1. Sintaxis básica

FoxDot es una librería de Python que se apoya en SuperCollider para emitir sonido, y al tratarse de una librería de Python su sintaxis es muy similar a la del mismo. Cuenta con las mismas reglas de indentación, misma forma de declarar funciones y asignación de variables, pero también cuenta con elementos sintácticos propios con mucho potencial.
Para empezar, en la consola de FoxDot cuando pulsas Ctrl + Enter se ejecuta un bloque de código, esto son líneas que no están separadas por una línea en blanco

```
# Esto constituye el primer bloque de código
p1 >> pluck()
p2 >> bass()

# Esto es el segundo bloque de código
p3 >> pluck()
```

Para declarar variables constantes es tan simple como poner `const()` y entre los paréntesis poner el valor deseado

```
c = const(10) # Ahora c no puede ser modificado de ninguna forma
```

2. SynthDef

Una de las formas de emitir sonido desde FoxDot es creando un objeto de tipo SynthDef (instrumento virtual de SuperCollider) de la siguiente forma:

```
p1 >> pluck() # Creo el objeto p1 que tiene asociado el SynthDef "pluck"
p1.stop() # Para detener el instrumento
```

La asignación de cualquier objeto Player (en este caso SynthDef) se realiza con el operador `>>` y no con =
Se puede usar la función `print(SynthDef)` para mostrar en la consola todos los instrumentos disponibles.
Un SynthDef cuenta con muchos atributos que se pueden mostrar usando `print(Player.get_attributes)`, siendo los más relevantes:

```
p1.degree = [0, 1, 2, 3] # Indica el grado de la escala, siendo 0 la tónica
p1.oct = [4, 5, 6] # Indica la octava de una nota, siendo 5 el valor por defecto
p1.dur = [1, 1/2, 0.25] # Indica la duración de las notas, siendo 1 una negra
p1.scale = Scale.minor # Indica la escala, siendo C Major la predeterminada
p1.amp = [1, 2, 1/2, 0] # Indica la amplitud de una nota, siendo 1 el valor por defecto
p1.bpm = 100 # Indica el tempo del objeto (ignorando el tempo del propio reloj)
p1.pan = [-1, 0, 1] # Indica el paneo de la nota, donde 0 es enviar equitativamente el audio entre ambos canales
p1.sus = [1, 1/2] # Indica la duración de la nota, por defecto es igual al valor de dur
```

Los atributos se pueden cambiar uno a uno manualmente como es mostrado anteriormente, o bien se pueden inferir al momento de crear un objeto de la siguiente forma:

```
p1 >> pluck([0, 1, 2, 3], oct=[4, 5, 6], dur=[1, 1/2, 0.25])
```

El atributo *degree* no necesita de una asignación específica
Para mostrar todas las escalas que hay se puede usar la función `print(Scale.names())`
Se explicará más adelante el por qué ocurre esto, pero poner `p1 >> pluck((0, 2, 4))` es distinto que poner `p1 >> pluck([0, 2, 4])`, en resumidas cuentas los elementos entre paréntesis se procesan a la vez y los elementos entre corchetes se procesan individualmente

3. Sample

Un Sample es un tipo especial de SynthDef, salvo que en vez de tener una lista de números asociados a los grados de la escala, en este caso se tiene un string donde cada caracter se asocia a un sample de audio. Para mostrar todos los Sample disponibles se puede usar la función `print(Samples)` y la forma de crearlos es análoga a un SynthDef:

```
d1 >> play("x-o-")
```

En el ejemplo, **x** se asocia a un bombo, **o** se asocia a una caja y **-** se asocia a un hi-hat
Si no se quiere reproducir un sonido en un beat particular se deja un espacio en blanco
En los Sample se pueden envolver partes del string entre distintos delimitadores para darle más variedad a las secuencias:

- **Paréntesis**: Alterna en cada loop el sonido que se va a emitir en la secuencia

  ```
  d1 >> play("(x-)(-x)o-")
  ```

  Ese ejemplo sería equivalente a poner `d1 >> play("x-o--xo-")`, pues en el primer loop tomaría de ambas secuencias entre paréntesis **x** y **-** respectivamente, mientras que en el segundo loop se toma la que queda pendiente, que sería **-** y **x**, y así sucesivamente

- **Corchetes**: Reproduce la secuencia delimitada en un único beat

  ```
  d1 >> play("x-o[---]")
  ```

  En ese ejemplo se reproducen tres hi-hat seguidos en cada cuarto beat, y cuanto mayor sea el tempo, más rápido se reproducen

- **Llaves**: Reproduce en cada loop uno de los samples de manera aleatoria

  ```
  d1 >> play("x-o{ox=}")
  ```

  En ese ejemplo en cada cuarto beat se puede reproducir aleatoriamente un bombo, una caja o un platillo, que tiene asociado el símbolo =

- **Mayor menor**: Esto ahorra el crear varios objetos Sample, pues se encarga de reproducir dos o más secuencias a la vez

  ```
  d1 >> play("<x-o-><  + + [ +]>")
  ```

  Ese ejemplo sería equivalente a ejecutar el siguiente bloque de código:

  ```
  d1 >> play("x-o-")
  d2 >> play("  + + [ +]")
  ```

Naturalmente las cuatro técnicas mencionadas anteriormente se pueden combinar entre sí para crear ritmos mucho más complejos

4. Pattern

En Python los operadores de + o * aplicados a una lista tienen un comportamiento enfocado en listas y no con un enfoque aritmético, por ejemplo:

```
print([1, 2, 3] + 2) # Esto produce la lista [1, 2, 3, 2]
print([1, 2, 3] * 2) # Esto produce la lista [1, 2, 3, 1, 2, 3]
```

Si queremos hacer una operación aritmética en todos los elementos de una lista tendríamos que hacerlo mediante recorridos sobre listas o listas intensionales. FoxDot arregla esto mediante el objeto Pattern, que es una lista a la cual si se le aplica una operación aritmética realiza dicha operación en todos los elementos del Pattern, entonces reescribiendo los ejemplos anteriores con Pattern se quedan así:

```
print(P[1, 2, 3] + 2) # Esto produce el Pattern P[3, 4, 5]
print(P[1, 2, 3] * 2) # Esto produce el Pattern P[2, 4, 6]
```

Para crear un objeto Pattern se puede usar el propio constructor del objeto Pattern de la siguiente forma: `pt1 = Pattern([0, 1, 2])` o, más fácil, poner una P al inicio de una lista como sigue: `pt1 = P[0, 1, 2]`. Aquí la asignación se hace como en Python de toda la vida con el operador = y no con `>>` como vimos antes con los SynthDef

Una operación aritmética entre dos Pattern `pt1` y `pt2` lo que hace es iterar sobre ambos Pattern simultáneamente una cantidad de veces igual al m. c. m.$(len(pt1), len(pt2))$, por ejemplo:

```
print(P[1, 2, 3] + P[4, 5]) # Esto produce el Pattern P[5, 7, 7, 6, 6, 8]
print(P[1, 2, 3, 4] * P[2, 3]) # Esto produce el Pattern P[2, 6, 6, 12]
```

En el primer caso, como $\text{m. c. m.}(2, 3) = 6$ entonces va a iterar seis veces emparejando con el elemento que le toca, desarrollando todas las iteraciones serían: (1+4), (2+5), (3+4), (1+5), (2+4) y (3+5)
En el segundo caso, como $\text{m. c. m.}(2, 4) = 4$ entonces va a iterar cuatro veces emparejando con el elemento que le toca, pero en este caso no se realizan todas las combinaciones, desarrollando todas las iteraciones serían: (1*2), (2*3), (3*2) y (4*3)

El acceso a los elementos de un Pattern es similar al de Python, en ambos casos se usa el operador [ ], pero mientras que en Python puedes usar tanto índices positivos como negativos para acceder a los elementos de una lista, estos índices siempre deben estar dentro de los límites del tamaño de la lista

```
l = [0, 1, 2]
print(l[1]) # 1
print(l[-1]) # 2
print(l[3]) # Error
```

En FoxDot el acceso siempre devuelve un elemento, debido a que el acceso en los Pattern son modulo-indexables

```
p = P[0, 1, 2]
print(p[1]) # 1
print(p[-1]) # 2
print(p[3]) # 0
```

Como en los Pattern el operador + tiene un significado aritmético y no es la concatenación como en Python, aquí la concatenación entre dos Pattern se hace con el operador |

```
print(P[1, 2, 3] | [4, 5, 6] | 7) # Esto produce el Pattern P[1, 2, 3, 4, 5, 6, 7]
```

Importante matiz en como los elementos posteriores no eran Pattern, sin embargo el resultado final se mantuvo como un Pattern, esto se debe a que todo objeto que sea concatenado a un Pattern como base entonces dará como resultado un Pattern

Similar al range de Python, en FoxDot puedes usar la función `PRange` para crear un Pattern con un orden secuencial y un rango específico

```
print(PRange(5)) # Esto produce el Pattern P[0, 1, 2, 3, 4]
print(PRange(0, 10, 2)) # Esto produce el Pattern P[0, 2, 4, 6, 8]
print(p[3]) # 0
```

La anidación de Pattern es peculiar

```
pt1 = P[0, 1, [2, 3]]
print(pt1) # Esto produce el Pattern P(0, 1, P[2, 3])
```

Para empezar, toda lista que esté anidada en un Pattern será transformada automáticamente en un Pattern, pero si intentamos acceder al tercer elemento de la lista

```
print(pt1[2]) # 2
```

No devuelve el Pattern anidado entero, sino el primer elemento del Pattern anidado, el motivo detrás de todo esto es el *lacing*, que internamente produce el siguiente Pattern: `P[0, 1, 2, 0, 1, 3]`, además si mostramos la longitud de `pt1` nos devolverá 6
Si queremos agrupar varios elementos en uno solo para eso se usa el objeto PGroup

```
pt2 = P[0, 1, (2, 3)]
print(pt2) # Esto produce el Pattern P(0, 1, P(2, 3))
```

Si en vez de anidar los valores con corchetes lo hacemos con paréntesis se crea un PGroup, por tanto si ahora accedemos al tercer elemento de la lista

```
print(pt2[2]) # P(2, 3)
```

Ahora sí nos devuelve los dos objetos como uno solo, además de que la longitud de `pt2` se mantiene en 3

4.1. Métodos de un Pattern

Los Pattern al igual que las listas de Python cuenta con métodos para modificar el propio Pattern, para llamar a cada método es tan simple como poner un . tras el Pattern y después el método al que quieres llamar. Aquí hay algunos de esos métodos

- `shuffle(n)`: Devuelve los contenidos del Pattern ordenados aleatoriamente y en tantas permutaciones como `n` indique

  ```
  print(P[0, 1, 2, 3].shuffle()) # P[0, 3, 2, 1]
  print(P[0, 1, 2, 3].shuffle(2)) # P[3, 1, 2, 0, 2, 1, 3, 0]
  ```

  El tamaño del Pattern resultante es $n * len(Pattern)$

- `reverse()`: Devuelve los contenidos del Pattern pero con el orden invertido

  ```
  P[0, 1, 2, 3].reverse() # P[3, 2, 1, 0]
  P[[0, 4], 3, 2, 1].reverse() # P[1, 2, 3, P[0, 4]]
  ```

  Este método no invierte el orden de cualquier elemento anidado

- `mirror()`: Hace lo mismo que `reverse()` pero esta vez invierte el orden de cualquier elemento anidado

  ```
  P[[0, 4], 3, 2, 1].mirror() # P[1, 2, 3, P[4, 0]]
  ```

- `stutter(n)`: Devuelve un Pattern con cada elemento repetido `n` veces, si `n` es un Pattern entonces se va alternando entre los distintos elementos de `n`

  ```
  print(P[0, 1, 2, 3].stutter(3)) # P[0, 0, 0, 1, 1, 1, 2, 2, 2, 3, 3, 3]
  print(P[0, 1, 2, 3].stutter([1, 2])) # P[0, 1, 1, 2, 3, 3]
  print(P[0, 1].stutter([2, 2, 3])) # P[0, 0, 1, 1, 0, 0, 0]
  ```

  En el segundo caso se va iterando en ambos Pattern hasta que se terminen los elementos del Pattern de mayor longitud, entonces el 0 lo repite 1 vez, el 1 lo repite 2 veces, el 2 lo repite 1 vez y el 3 lo repite 2 veces

- `arp(seq)`: Aplica un stutter en base a la longitud de `seq`, y posteriormente le suma al valor base los valores descritos en `seq` (útil para arpegiar)

  ```
  print(P[0, 1, 2, 3].arp([0, 4])) # P[0, 4, 1, 5, 2, 6, 3, 7]
  ```

  En este caso sería primero hacer un `stutter(2)` para duplicar cada elemento y después a la i-ésima repetición se le suma el i-ésimo elemento de `seq`, por ejemplo con el 0 se duplicaría dejando 0, 0 pero al ser el segundo elemento de `seq` un 4 se le suma al segundo 0 dicho valor, quedando 0, 4

- `transform(f)`: Devuelve el resultado de aplicar la función `f` a todos los elementos del Pattern

  ```
  def is_even(n):
      return 1 if n % 2 == 0 else 0

  p = P[1,2,3,4,5,6,7,8,9].transform(is_even)
  print(p) # P[0, 1, 0, 1, 0, 1, 0, 1, 0]
  ```

Hay muchos más métodos descritos aquí

4.2. Funciones de un Pattern

Las funciones son formas útiles de generar Patterns

- `PStep(n, value, default=0)`: Crea un Pattern de `n` elementos asignados a `default`, exceptuando el último valor al que se le asigna `value`

  ```
  print(PStep(5, 4)) # P[0, 0, 0, 0, 4]
  print(PStep(3, "o", "x")) # P["x", "x", "o"]
  ```

- `PEuclid(n, k)`: Emplea el algoritmo de Euclides para crear un Pattern de longitud `k` con `n` pulsos a dividir de la forma más equitativa posible

```
print(PEuclid(2, 8)) # P[1, 0, 0, 0, 1, 0, 0, 0]
print(PEuclid(3, 8)) # P[1, 0, 0, 1, 0, 0, 1, 0]
print(PEuclid(4, 8)) # P[1, 0, 1, 0, 1, 0, 1, 0]
```

  Un 1 simboliza que ahí se debe dar un pulso, y un 0 es un silencio

- `PDur(n, k, start=0, dur=0.25)`: Aplica la función `PEuclid` pero en vez de devolver un Pattern de 0 y 1 devuelve la duración de cada uno de los `n` pulsos

```
print(PDur(3, 8)) # P[0.75, 0.75, 0.5]
```

  Si comparamos este ejemplo con `PEuclid(3, 8)` da un resultado coherente, pues podemos traducir el output de P[1, 0, 0, 1, 0, 0, 1, 0] a P[3, 3, 2] que es el número de beats que hay entre cada pulso, si a esos valores le multiplicamos el valor del parámetro `dur` de la función nos queda [0.75, 0.75, 0.5]

Hay muchas más funciones descritas aquí

4.3. Generadores de un Pattern

Los generadores de Patterns son similares a las funciones vistas anteriormente, salvo que los generadores producen Patterns de rango infinito, donde los valores no se quedan almacenados en memoria sino que se obtienen en el momento que se soliciten

```
pt_gen = PRand(0, 10)
print(pt_gen) # PRand(0, 10)
print(pt_gen[0]) # 7
print(pt_gen[:5]) # P[7, 4, 3, 1, 1]
```

Es por eso que cuando se intenta imprimir un Pattern creado a través de un generador no muestra ningún valor pero cuando accedo a cualquiera de sus índices sí que devuelve un valor, o como en Python puedes hacer uso del slicing para devolver un subpatrón del propio generador Al tratarse de un Pattern puedes aplicar operaciones aritméticas sobre el Pattern o funciones como `transform`
Estos son algunos de los generadores disponibles:

- `PRand(lo, hi)`: Devuelve un Pattern de valores aleatorios entre [`lo`, `hi`]. Si en vez de pasar dos valores pasas una lista de valores entonces formará el Pattern tomando valores de esa lista

```
print(PRand(0, 10)[:5]) # P[8, 10, 4, 0, 2]
print(PRand([0,1,2])[:5]) # P[1, 0, 0, 2, 1]
```

- `PWhite(lo, hi)`: Análogo al anterior, pero los valores son en punto flotante (no se puede pasar una lista como argumento)

```
print(PWhite(0, 10)[:3]) # P[3.0836771546951747, 3.125579463761354, 6.1583227229147 24]
```

- `PSquare()`: Devuelve la serie de cuadrados perfectos

```
print(PSquare()[:5]) # P[0, 1, 4, 9, 16]
```

- `PFibMod()`: Devuelve la sucesión de Fibonacci

```
print(PFibMod()[:7]) # P[0, 1, 1, 2, 3, 5, 8]
```

Hay muchos más generadores descritos aquí

5. PGroup

Como se ha mencionado anteriormente, los PGroup sirven para agrupar varios elementos, esta agrupación permite reproducir todos sus elementos a la vez (útil para crear acordes) Similar a los Pattern, para crear un PGroup basta con poner una P al principio de una tupla de Python
Si aplicas una operación entre un Pattern y un PGroup, esto genera un Pattern de PGroups donde el i-ésimo PGroup es obtenido de hacer la operación con cada uno de los elementos del PGroup y el i-ésimo elemento del Pattern

```
print(P[2, 3, 4] * (1, 2)) # Esto produce el Pattern P[P(2,4), P(3,6), P(4,8)]
```

En el ejemplo, el PGroup P(2,4) sale de tomar el 2 del Pattern y multiplicarlo con cada elemento del PGroup (1,2), y así con el resto de elementos del Pattern
Dentro de los PGroup hay un grupo de PGroups llamado Extended PGroup o PGroup Prime, que consiste en un PGroup al que se le añade un operador matemático para activar un comportamiento concreto si el PGroup es usado en un Player. Los distintos PGroup Prime que hay son:

- **PGroup-Star**: Reparte de manera equitativa los contenidos del PGroup en base al valor de dur (esencialmente un arpegiador)

  ```
  p1 >> pluck(P(0, 2, 4), dur=2)
  p2 >> pluck(P*(0, 2, 4), dur=2)
  ```

  En el objeto `p1` al usar un PGroup normal se reproduce un acorde, pero en el caso de `p2` al usar P* va a arpegiar dicho acorde, reproduciendo cada nota individualmente con la misma duración, donde dicha duración se calcula de dividir el valor de dur entre el número de elementos de P* (en este caso cada nota tendría un dur de $2/3 \approx 0.67$)

- **PGroup-Plus**: Hace lo mismo que PGroup-Star pero actúa sobre el valor de sus en vez del valor de dur

  ```
  p1 >> pluck(P+(0, 2, 4, 6), dur=PDur(3,8), sus=1)
  ```

- **PGroup-Double Star**: Hace lo mismo que PGroup-Star pero reproduce las notas de manera aleatoria

  ```
  p1 >> pluck(P[0, 1, 2, P**(3, 5, 7, 9)])
  ```

- **PGroup-Div**: Hace lo mismo que el PGroup-Star, pero en cada loop alterna entre arpegiar el acorde y reproducir el acorde simultáneamente

  ```
  p1 >> pluck(P[0, 1, 2, P/(3, 5, 7, 9)])
  ```

- **PGroup-Pow**: Utiliza el último valor del propio grupo para forzar un delay de dicho valor, independientemente del valor de dur o sus

  ```
  p1 >> pluck([0, 2, P^(3, 5, 7, 0.5)], dur=1, sus=4)
  ```

  En ese ejemplo cada nota que se toque va a tocarse una segunda vez tras un delay de 0.5